

« Une donnée mal modélisée corrompt tout ce qu'on construit dessus. Une donnée mal protégée détruit tout ce qu'on a construit. »

CHAPITRE I – FONDAMENTAUX DE LA MODÉLISATION

— POURQUOI MODÉLISER AVANT DE CODER —

La modélisation est la phase où on décide de la structure de la réalité qu'on va capturer en données. Une mauvaise décision à ce stade se paye à chaque requête, à chaque migration, à chaque évolution du système pendant des années.

Un modèle de données répond à trois questions fondamentales. Quelles sont les entités du domaine – les objets réels que le système doit représenter ? Quelles sont leurs attributs – les informations qu'on doit conserver sur chacune ? Quelles sont leurs relations – comment ces entités interagissent-elles entre elles ?

Un bon modèle reflète fidèlement le métier, pas la technologie. Le choix du langage de programmation ou du SGBD vient après. Le modèle conceptuel est indépendant de l'implémentation. Confondre les deux est l'erreur la plus coûteuse en conception de systèmes d'information.

La règle de la granularité : une entité doit représenter exactement un concept du domaine. Ni plus, ni moins. Une entité "ClientCommande" qui mélange les informations client et les informations commande est un signe que le modèle n'a pas été pensé suffisamment.

— LES NIVEAUX DE MODÉLISATION —

Le niveau conceptuel décrit le quoi sans s'occuper du comment. Il capture les entités, leurs attributs, et leurs relations dans le langage du métier. Il est compréhensible par un responsable métier non technique. C'est ici que les décisions fondamentales se prennent.

Le niveau logique traduit le modèle conceptuel en structures adaptées à un type de base de données – relationnel, document, graphe. Il introduit les clés, les tables de liaison, les contraintes d'intégrité. Il est compréhensible par un développeur.

Le niveau physique correspond à l'implémentation réelle dans un SGBD précis. Il prend en compte les performances, les index, le partitionnement, les types de données spécifiques au moteur. C'est le niveau du DBA.

Ces trois niveaux doivent rester cohérents entre eux. Une modification du modèle conceptuel implique une révision du logique et potentiellement du physique. Sauter le niveau conceptuel pour aller directement au physique produit des systèmes qui répondent à la technique mais pas au métier.

CHAPITRE II – MERISE

— LA MÉTHODE MERISE —

Merise est une méthode française de conception des systèmes d'information. Elle sépare les données des traitements et structure la conception en niveaux successifs. Elle reste très utilisée dans les grandes organisations françaises et les administrations. Comprendre Merise, c'est comprendre le langage de conception d'une large part des SI existants.

Son principe central est la séparation nette entre ce que le système doit mémoriser (les données) et ce que le système doit faire (les traitements). Cette séparation évite de mélanger la structure et la logique, deux choses qui évoluent à des rythmes différents.

— LE MODÈLE CONCEPTUEL DES DONNÉES (MCD) —

Le MCD est le cœur de Merise. Il représente les entités du domaine, leurs attributs, et les associations qui les relient, indépendamment de toute considération technique.

Une entité est un objet du monde réel qu'on doit décrire. CLIENT, COMMANDE, PRODUIT, FOURNISSEUR sont des entités. Chaque entité a un identifiant unique et des attributs qui la décrivent. L'identifiant est souligné dans le schéma.

CLIENT (#id_client, nom, prenom, email, date_inscription)
COMMANDE (#id_commande, date_commande, statut, montant_total)
PRODUIT (#id_produit, libelle, prix_unitaire, stock)

Une association relie deux ou plusieurs entités et exprime une relation du monde réel. PASSER relie CLIENT et COMMANDE. CONTENIR relie COMMANDE et PRODUIT. Les associations peuvent avoir leurs propres attributs.

Les cardinalités expriment combien d'occurrences d'une entité peuvent être liées à une occurrence de l'autre. Elles se lisent de chaque côté de l'association.

(1,1) signifie exactement une occurrence.
(0,1) signifie zéro ou une occurrence.
(1,n) signifie une ou plusieurs occurrences.
(0,n) signifie zéro ou plusieurs occurrences.

Un client passe entre 0 et n commandes : (0,n) côté CLIENT.
Une commande est passée par exactement 1 client : (1,1) côté COMMANDE. Cette lecture se fait toujours depuis l'entité vers l'association.

Les cardinalités déterminent directement la structure de la base de données. Une relation (1,n)-(1,1) se traduit par une clé étrangère dans la table du côté (1,1). Une relation (0,n)-(0,n) se traduit par une table de liaison.

L'entité faible dépend d'une autre entité pour exister. Une LIGNE_COMMANDE ne peut exister sans une COMMANDE. Son identifiant est composé de son propre numéro et de l'identifiant de l'entité dont elle dépend.

— LE MODÈLE LOGIQUE DES DONNÉES (MLD) —

Le MLD traduit le MCD en tables relationnelles selon des règles déterministes. La traduction est mécanique si le MCD est correct.

Règle 1 – Chaque entité devient une table. Ses attributs deviennent des colonnes. Son identifiant devient la clé primaire.

Règle 2 – Une association (0,n)-(0,n) ou (1,n)-(1,n) devient une table de liaison avec les clés primaires des deux entités comme clé composée. Les attributs de l'association deviennent des colonnes de cette table.

Règle 3 – Une association (1,1)-(0,n) ou (1,1)-(1,n) ne crée pas de table supplémentaire. La clé de l'entité du côté (0,n) migre comme clé étrangère dans la table du côté (1,1).

APPLICATION :

CLIENT(id_client PK, nom, prenom, email, date_inscription)
COMMANDE(id_commande PK, date_commande, statut, montant_total, id_client FK)
PRODUIT(id_produit PK, libelle, prix_unitaire, stock)
CONTENIR(id_commande FK, id_produit FK, quantite, prix_ligne)
PK composée (id_commande, id_produit)

— LA NORMALISATION —

La normalisation est le processus d'organisation des données pour réduire la redondance et améliorer l'intégrité. Elle s'applique après la traduction MCD→MLD.

La première forme normale (1NF) exige que chaque colonne contienne une valeur atomique – une seule valeur, pas une liste. Un champ "téléphones" qui contient "0601, 0602" viole la 1NF. Il faut une table TELEPHONE séparée.

La deuxième forme normale (2NF) s'applique aux tables avec clé composée. Chaque colonne non clé doit dépendre de la totalité de la clé, pas d'une partie. Dans la table CONTENIR, le libelle du produit ne dépend que de id_produit, pas du couple (id_commande, id_produit). Il doit être dans la table PRODUIT.

La troisième forme normale (3NF) exige qu'aucune colonne non clé ne dépende d'une autre colonne non clé. Si la table CLIENT contient id_ville et nom_ville, et que nom_ville dépend de id_ville, il faut une table VILLE séparée.

En pratique, la 3NF est le niveau de normalisation cible pour la plupart des systèmes transactionnels. Une dénormalisation partielle peut être justifiée dans les systèmes décisionnels pour des raisons de performance.

— LE MODÈLE CONCEPTUEL DES TRAITEMENTS (MCT) —

Le MCT décrit ce que le système fait – les événements qui déclenchent des opérations, les opérations elles-mêmes, et les résultats qu'elles produisent.

Un événement est un fait externe qui déclenche une réaction du système : une commande reçue, un paiement effectué, un stock atteint le seuil minimum.

Une opération est un ensemble d'actions déclenchées par un ou plusieurs événements. Elle est atomique – soit elle s'exécute entièrement, soit elle ne s'exécute pas.

Un résultat est ce que l'opération produit : un document, une mise à jour d'état, un déclenchement d'un autre événement.

Les règles d'émission définissent sous quelles conditions les résultats sont produits. Toujours, ou conditionnellement selon l'état du système.

CHAPITRE III – UML

— PRÉSENTATION D'UML —

UML – Unified Modeling Language – est un langage de modélisation universel orienté objet. Contrairement à Merise qui est une méthode, UML est un langage : il décrit mais ne prescrit pas de démarche. Il propose 14 types de diagrammes organisés en

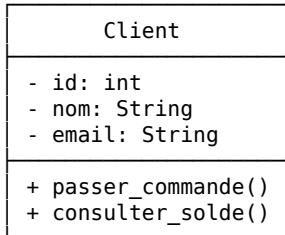
deux familles : les diagrammes structurels et les diagrammes comportementaux.

Pour l'analyse de données et les systèmes d'information, cinq diagrammes UML sont essentiels : le diagramme de classes, le diagramme de cas d'utilisation, le diagramme de séquence, le diagramme d'activité, et le diagramme de déploiement.

— LE DIAGRAMME DE CLASSES —

C'est l'équivalent UML du MCD Merise. Il représente les classes du domaine avec leurs attributs, leurs méthodes, et leurs relations. C'est le diagramme le plus utilisé en conception de systèmes d'information.

Une classe est représentée par un rectangle en trois parties : le nom en haut, les attributs au milieu, les méthodes en bas.



Les visibilitées : + public (accessible partout), - privé (accessible uniquement dans la classe), # protégé (accessible dans la classe et ses sous-classes).

Les relations entre classes ont des sémantiques précises.

L'association est la relation la plus générale. Un Client passe des Commandes. Elle porte une cardinalité aux deux extrémités. Client "1" passe "0..*" Commandes – un client passe zéro ou plusieurs commandes, une commande est passée par exactement un client.

L'agrégation exprime une relation "contient" non exclusive. Un Département contient des Employés, mais un Employé peut exister sans Département. Symbolisée par un losange vide.

La composition exprime une relation "contient" exclusive et existentielle. Une Commande contient des LignesCommande, et une LigneCommande ne peut exister sans sa Commande. Si la Commande est supprimée, ses lignes le sont aussi. Symbolisée par un losange plein.

L'héritage exprime une relation "est un". Un ClientProfessionnel est un Client. Il hérite de tous les attributs et méthodes de Client et peut en ajouter. Symbolisé par une flèche à pointe triangulaire vide.

La dépendance exprime qu'une classe utilise une autre de façon passagère. Symbolisée par une flèche en pointillé.

— LE DIAGRAMME DE CAS D'UTILISATION —

Il représente les fonctionnalités du système du point de vue de ses utilisateurs. Il répond à la question : qui fait quoi avec le système ?

Les acteurs sont les entités externes qui interagissent avec le système : utilisateurs humains, autres systèmes, appareils. Ils sont représentés par des bonhommes.

Les cas d'utilisation sont les fonctionnalités : Passer une commande, Consulter l'historique, Générer un rapport. Représentés par des ellipses.

Les relations structurent les cas d'utilisation. Include signifie qu'un cas d'utilisation en inclut systématiquement un autre. Extend signifie qu'un cas étend un autre dans certaines conditions. Héritage signifie qu'un acteur hérite des cas d'utilisation d'un autre.

Ce diagramme est le point d'entrée de toute analyse. Il garantit que ce qu'on construit répond aux besoins réels des utilisateurs, pas aux suppositions des concepteurs.

— LE DIAGRAMME DE SÉQUENCE

Il représente l'ordre chronologique des interactions entre objets ou composants pour réaliser un cas d'utilisation. Il répond à la question : dans quel ordre les choses se passent-elles ?

Les lignes de vie sont des colonnes verticales représentant chaque participant (objet, composant, acteur).

Les messages sont des flèches horizontales entre les lignes de vie. Un message synchrone attend une réponse avant de continuer. Un message asynchrone n'attend pas.

Les fragments combinés gèrent les structures de contrôle : alt pour les alternatives (if/else), loop pour les boucles, opt pour les optionnels.

Exemple pour "Passer une commande" :
Client → Interface : saisir commande
Interface → ServiceCommande : valider commande
ServiceCommande → StockService : vérifier disponibilité
StockService → ServiceCommande : stock OK
ServiceCommande → PaiementService : débiter
PaiementService → ServiceCommande : paiement confirmé
ServiceCommande → Interface : confirmation commande

— LE DIAGRAMME D'ACTIVITÉ

Il représente le flux d'un processus ou d'un algorithme. C'est l'équivalent UML du logigramme. Il répond à la question : comment le processus se déroule-t-il, avec quelles décisions et quels parallélismes ?

Utile pour modéliser les processus métier avant de les automatiser. Un processus bien modélisé en diagramme d'activité se traduit directement en code propre.

— LE DIAGRAMME DE DÉPLOIEMENT

Il représente l'infrastructure sur laquelle le système s'exécute : serveurs, conteneurs, réseaux, composants déployés. Il répond à la question : où tourne le système et comment les composants communiquent-ils ?

Indispensable pour documenter une architecture microservices, une architecture cloud, ou toute infrastructure distribuée.

CHAPITRE IV – ARCHITECTURE DES DONNÉES ERP

— CE QU'EST UN ERP

Un ERP – Enterprise Resource Planning ou Progiciel de Gestion Intégré – est un système qui centralise et intègre les données de toutes les fonctions d'une entreprise dans une base de données commune. Finance, comptabilité, achats, ventes, production, RH, logistique – tout partage le même référentiel.

L'intégration est la valeur centrale d'un ERP. Une commande saisie par le commercial met à jour automatiquement le stock, génère une facture, affecte les comptes comptables, et déclenche un bon de livraison logistique. Sans ERP, ces quatre actions se font dans quatre systèmes différents avec des ressaisies et des délais entre chacune.

Les ERP majeurs du marché sont SAP (référence absolue dans les grandes entreprises), Oracle (concurrent direct de SAP), Microsoft Dynamics (PME et ETI), Sage (très présent en France), et Odoo (open source, en forte croissance dans les PME).

— ARCHITECTURE TECHNIQUE D'UN ERP —

La base de données d'un ERP repose sur un modèle relationnel hyper-normalisé avec des centaines à des milliers de tables. SAP possède des dizaines de milliers de tables. Cette complexité reflète la richesse fonctionnelle du système.

L'architecture standard d'un ERP est en trois couches. La couche présentation est l'interface utilisateur – navigateur web, client lourd, application mobile. La couche applicative contient la logique métier – les modules fonctionnels et les règles de gestion. La couche données contient la base de données centrale.

Les modules fonctionnels correspondent aux domaines métier. Chaque module a son propre ensemble de tables mais partage les tables de référentiel communes. Un module Ventes et un module Comptabilité partagent la même table CLIENT et la même table DEVISE.

Les tables de base d'un ERP se divisent en plusieurs catégories.

Les tables de référentiel contiennent les données stables qui servent de référence à tout le système : clients, fournisseurs, produits, plan comptable, centres de coût, devises, pays. Ces tables sont partagées par tous les modules. Leur cohérence est fondamentale.

Les tables transactionnelles contiennent les mouvements : commandes, factures, écritures comptables, mouvements de stock, feuilles de temps. Elles sont volumineuses et indexées sur les dates et les identifiants de référentiel.

Les tables de paramétrage définissent le comportement du système : règles de calcul, workflows d'approbation, modèles de documents, droits d'accès. Elles varient selon la configuration de l'entreprise.

— LE RÉFÉRENTIEL MAÎTRE —

Le référentiel maître – Master Data Management – est le cœur de toute architecture ERP. Il centralise les données de base partagées par tous les modules et garantit leur unicité.

Le référentiel client contient un enregistrement unique par client avec un identifiant unique. Ce même client est vu par les ventes, la comptabilité, le service client, et la logistique. Sans référentiel maître, chaque département crée son propre enregistrement client avec ses propres données et ses propres erreurs.

Le référentiel produit contient la description technique, commerciale, logistique, et comptable de chaque article. La fiche produit est l'objet le plus riche d'un ERP. Elle peut contenir des centaines d'attributs selon le secteur.

La qualité du référentiel maître détermine la qualité de toutes les analyses. Des données de référentiel incohérentes produisent des analyses incohérentes peu importe la sophistication des outils analytiques.

— INTÉGRATION ENTRE ERP ET SYSTÈMES EXTERNES —

Un ERP moderne ne vit pas seul. Il s'intègre avec des systèmes spécialisés : CRM pour la gestion commerciale, WMS pour la gestion d'entrepôt, MES pour la production, BI pour l'analyse décisionnelle, e-commerce pour les ventes en ligne.

Les patterns d'intégration sont standardisés. L'API REST est le standard actuel pour les intégrations temps réel. Les fichiers d'échange – XML, JSON, EDI – restent utilisés pour les échanges B2B et les intégrations batch. L'ESB (Enterprise Service Bus) est l'architecture d'intégration classique des grandes organisations.

L'ETL – Extract Transform Load – est le mécanisme central d'alimentation des entrepôts de données depuis l'ERP. Les données transactionnelles de l'ERP sont extraites, transformées pour l'analyse, et chargées dans un Data Warehouse où elles sont interrogées par les outils BI.

En mission freelance, accéder aux données d'un ERP se fait généralement via une vue SQL exposée par l'ERP, une API officielle du progiciel, ou un export CSV/Excel produit par un rapport standard. L'accès direct aux tables de l'ERP est possible mais risqué – les schémas sont complexes, les noms de tables cryptiques, et une requête mal écrite peut impacter les performances du système de production.

— INTERROGER UN ERP EN PYTHON —

La connexion à la base de données de l'ERP suit le même pattern que toute base SQL :

```
engine = create_engine("mssql+pyodbc://user:pass@server/db")
```

Les tables clés à connaître dans un ERP générique :

- La table clients contient id_client, code_client, raison_sociale, categorie, code_commercial, date_creation.
- La table commandes contient id_commande, id_client, date, statut, montant_ht, montant_ttc, id_commercial.
- La table lignes_commande contient id_ligne, id_commande, id_produit, quantite, prix_unitaire, remise, montant_ligne.
- La table produits contient id_produit, code_article, libelle, famille, prix_vente, cout_revient, stock_disponible.
- La table ecritures_comptables contient id_ecriture, date, compte, libelle, debit, credit, id_piece.

La jointure centrale de tout reporting commercial :

```
SELECT c.raison_sociale, SUM(lc.montant_ligne) as ca
FROM commandes co
JOIN clients c ON co.id_client = c.id_client
JOIN lignes_commande lc ON co.id_commande = lc.id_commande
WHERE co.date BETWEEN :debut AND :fin
AND co.statut = 'Validée'
GROUP BY c.raison_sociale
ORDER BY ca DESC
```

CHAPITRE V – SÉCURITÉ DES DONNÉES

— LES PRINCIPES FONDAMENTAUX —

La sécurité des données repose sur trois piliers fondamentaux connus sous l'acronyme CIA. La Confidentialité garantit que les données ne sont accessibles qu'aux personnes autorisées. L'Intégrité garantit que les données ne sont pas altérées de façon non autorisée. La Disponibilité garantit que les données sont accessibles quand elles sont nécessaires.

Ces trois piliers sont en tension permanente. Augmenter la confidentialité peut réduire la disponibilité. Augmenter

la disponibilité peut menacer la confidentialité. La sécurité est un équilibre, pas un absolu.

Le principe du moindre privilège est la règle d'or de la sécurité des données. Chaque utilisateur, chaque application, chaque processus ne doit avoir accès qu'aux données strictement nécessaires à sa fonction, et rien de plus. Un analyste qui n'a besoin que des données de vente n'a pas besoin d'accès aux données RH.

La défense en profondeur signifie qu'on ne compte pas sur un seul mécanisme de sécurité. Si un mécanisme échoue, un autre prend le relais. C'est l'application du principe qu'aucun système n'est infaillible.

— CLASSIFICATION DES DONNÉES —

Toutes les données ne méritent pas le même niveau de protection. La classification permet d'allouer les ressources de sécurité de façon proportionnelle à la sensibilité.

Les données publiques peuvent être diffusées librement. Les données internes sont réservées à l'organisation mais leur divulgation n'est pas critique. Les données confidentielles ont un impact significatif si divulguées. Les données secrètes ont un impact majeur ou catastrophique si divulguées.

Les données personnelles – qui permettent d'identifier une personne physique – sont soumises au RGPD en Europe. Leur traitement est encadré par des obligations légales avec des sanctions potentielles allant jusqu'à 4% du chiffre d'affaires mondial ou 20 millions d'euros.

Les données critiques pour l'activité méritent une attention particulière indépendamment de leur caractère personnel : données financières, propriété intellectuelle, secrets commerciaux, données de production.

— PROTECTION DES DONNÉES EN BASE —

Le chiffrement au repos protège les données stockées. Les bases de données modernes offrent le chiffrement natif du stockage. Les champs particulièrement sensibles – mots de passe, numéros de carte bancaire, numéros de sécurité sociale – méritent un chiffrement au niveau colonne en plus du chiffrement du disque.

Ne jamais stocker un mot de passe en clair. Toujours stocker son hash avec un algorithme adapté aux mots de passe – bcrypt, argon2, scrypt. Un hash MD5 ou SHA1 d'un mot de passe n'est pas une protection – ces algorithmes sont trop rapides et vulnérables aux attaques par dictionnaire.

Le chiffrement en transit protège les données pendant leur transfert. TLS 1.2 ou 1.3 est obligatoire pour toute connexion à une base de données accessible sur un réseau, a fortiori sur Internet. Une connexion non chiffrée expose toutes les données échangées à quiconque intercepte le trafic.

Les sauvegardes sont aussi des données sensibles. Une sauvegarde non chiffrée d'une base de données chiffrée anéantit la protection. Les sauvegardes doivent être chiffrées avec les mêmes exigences que la donnée source.

— CONTRÔLE D'ACCÈS —

L'authentification vérifie qu'un utilisateur est bien celui qu'il prétend être. Un mot de passe seul est insuffisant pour les systèmes sensibles. L'authentification à deux facteurs – quelque chose que tu connais et quelque chose que tu possèdes – est le minimum requis pour tout accès à des données sensibles.

L'autorisation contrôle ce qu'un utilisateur authentifié peut faire. Les rôles permettent de gérer les autorisations par groupe plutôt qu'individuellement. Un rôle ANALYSTE peut lire certaines tables. Un rôle ADMIN peut tout faire. Les utilisateurs héritent des droits de leurs rôles.

```
GRANT SELECT ON ventes TO analyste;
GRANT SELECT, INSERT, UPDATE ON ventes TO commercial;
GRANT ALL PRIVILEGES ON DATABASE TO admin;
```

La traçabilité des accès – qui a accédé à quoi et quand – est une exigence de sécurité et souvent une obligation légale. Les logs d'accès permettent de détecter les comportements anormaux et de reconstituer les événements en cas d'incident.

La révocation rapide des accès est aussi importante que leur attribution. Un employé qui quitte l'organisation doit perdre ses accès le jour de son départ, pas quand quelqu'un pense à le faire.

— SÉCURITÉ DES PIPELINES ET SCRIPTS PYTHON —

Ne jamais mettre de credentials dans le code source. Ni en clair, ni dans un commentaire, ni dans une variable codée en dur. Les credentials appartiennent aux variables d'environnement ou à un gestionnaire de secrets.

```
os.getenv("DB_PASSWORD") est correct.
password = "motdepasse123" dans le code est une catastrophe
qui attend de se produire, surtout si le code va sur GitHub.
```

Les injections SQL sont la première vulnérabilité des applications qui accèdent à des bases de données. Un paramètre utilisateur inséré directement dans une requête peut permettre à un attaquant d'exécuter n'importe quelle requête.

```
Vulnérable : query = f"SELECT * FROM users WHERE id={user_id}"
Sécurisé : query = text("SELECT * FROM users WHERE id=:id")
            result = conn.execute(query, {"id": user_id})
```

Valide toujours les données d'entrée avant de les utiliser dans une requête. Un identifiant censé être un entier doit être vérifié comme entier avant d'être utilisé. Ne fais jamais confiance à un paramètre externe.

Les logs ne doivent jamais contenir de données sensibles. Pas de mot de passe, pas de numéro de carte, pas de données personnelles dans les logs. Un log est souvent moins protégé que la base de données qu'il documente.

— RGPD ET OBLIGATIONS LÉGALES —

Le RGPD s'applique à toutes les organisations qui traitent des données de résidents européens, quelle que soit leur localisation. En tant que freelance qui traite des données client pour le compte d'une entreprise, tu es un sous-traitant au sens du RGPD. Tu as des obligations légales.

Les six bases légales du traitement sont : le consentement, l'exécution d'un contrat, l'obligation légale, la sauvegarde des intérêts vitaux, la mission d'intérêt public, et l'intérêt légitime. Chaque traitement doit reposer sur l'une d'elles.

Les droits des personnes concernées sont : le droit d'accès, le droit de rectification, le droit à l'effacement, le droit à la portabilité, le droit d'opposition, et le droit à la limitation du traitement. Le système d'information doit permettre de répondre à ces droits dans les délais légaux.

La minimisation des données impose de ne collecter que les données strictement nécessaires à la finalité déclarée. Collecter des données "au cas où elles seraient utiles un jour" est une violation du RGPD.

La durée de conservation doit être définie et respectée. Des données conservées indéfiniment sans base légale exposent l'organisation à un risque juridique. En pratique, une politique de purge automatique des données au-delà de leur durée de conservation légale ou contractuelle est la meilleure protection.

En cas de violation de données – accès non autorisé, perte, destruction – la notification à la CNIL est obligatoire dans les 72 heures si la violation présente un risque pour les personnes. La notification aux personnes concernées est obligatoire si le risque est élevé.

— BONNES PRATIQUES SYNTHÉTIQUES —

Chiffre les données sensibles au repos et en transit.

Applique le principe du moindre privilège à chaque accès.

Ne mets jamais de credentials dans le code source.

Utilise toujours des requêtes paramétrées, jamais de concaténation de chaînes dans les requêtes SQL.

Classe tes données par sensibilité et protège-les proportionnellement.

Journalise les accès aux données sensibles et surveille les comportements anormaux.

Révoque les accès immédiatement quand ils ne sont plus nécessaires.

Sauvegarde régulièrement. Teste régulièrement que les sauvegardes sont restaurables. Une sauvegarde non testée n'est pas une sauvegarde.

Maintiens un inventaire des données personnelles traitées et des bases légales associées.

Forme les équipes. La plupart des incidents de sécurité impliquent une erreur humaine. La technologie protège contre les attaquants. La formation protège contre les accidents.

VÉRITÉS DU TERRAIN

Un bon modèle de données est celui qu'on n'a plus besoin de changer quand le métier évolue normalement. Si chaque nouvelle fonctionnalité nécessite une restructuration profonde du modèle, le modèle était mauvais dès le départ.

Merise et UML ne s'opposent pas. Merise est plus adapté à la modélisation des données et à la culture française des SI. UML est plus adapté à la modélisation orientée objet et à la documentation de systèmes complexes. Sur un projet data, les deux ont leur place.

Les données d'un ERP sont les données les plus riches et les plus complexes qu'un analyste puisse rencontrer. Ce qui rend un analyste ERP précieux n'est pas la maîtrise du SQL – c'est la compréhension du métier derrière les tables.

La sécurité ajoutée après coup coûte dix fois plus cher que la sécurité conçue dès le départ. Intègre les exigences de sécurité dans la conception, pas dans la maintenance.

Un incident de sécurité n'est pas une question de si mais de quand. La question est de savoir si tu le détecteras avant que quelqu'un d'autre le détecte à ta place.

